

--- title: "WB-2026-15 · Data Stack para Agentes IA — Do Zero à Produção"
description: "Como projetar Postgres + Redis + BullMQ + Vector DB para um agente que escala. Cases reais de quem quebrou e quem deu certo."
tags: [webinar, data-stack, postgres, redis, vector-db, llm, multi-agent, escala] nivel: Master → Elite duracao: 90 min data: "2027-01-15" preletor: "Sir. Nexus Alencar (CTO) + convidado Qdrant" participacao: "live + replay"
pattern: "MMN_IA" --- **WB-2026-15 · Data Stack para Agentes IA — Do Zero à Produção** *Por trás de cada agente que "funciona em produção" tem um data stack sólido. Esta live mostra como projetar — da escolha do banco à estratégia de cache — com cases reais de quem quebrou e deu certo.* **Por Sir. Nexus Alencar · Academ'IA** Nexus Affil'IA'te · 2026 --- #

Sumário > **•** 1. Por que data stack é o gargalo #1 em produção > **•** 2. Os 4 tipos de dados em sistemas agentic > **•** 3. As 4 camadas: Postgres + Redis + BullMQ + Vector DB > **•** 4. Case 1: afiliado que quebrou (Postgres pra tudo) > **•** 5. Case 2: SaaS que escalou (data stack desde dia 1) > **•** 6. Postgres para multi-tenant (RLS, schemas, row-level) > **•** 7. Redis: cache, sessão, rate limit > **•** 8. BullMQ: filas assíncronas, retry, DLQ > **•** 9. Vector DB: Pinecone vs Qdrant vs Weaviate > **•** 10. Q&A com CTO + convidado Qdrant --- **1. Por que data stack é o gargalo #1 em produção** Em 2024, "construir o agente" era o gargalo. Em 2026, **rodar em escala** é o gargalo. **Sintomas clássicos:** - Latência sobe de 200ms (1 user) para 8s (100 users) - Custos de API explodem sem controle - Memória do agente "se perde" entre sessões - Agentes conflitam ao acessar mesmos dados - Impossível auditar o que cada agente fez **Causa:** data stack não foi projetado pra escala. Foi adicionado incrementalmente. **Solução:** planejar ANTES de construir o agente. --- **2. Os 4 tipos de dados em sistemas agentic** ### 1. Transacional (Postgres) Contas, planos, pedidos, billing, permissões — dados estruturados com ACID. ### 2. Sessão (Redis) Estado da conversa, contexto, histórico — key-value com TTL curto. ### 3. Assíncrono (BullMQ/SQS) Jobs pendentes, retries, webhooks — filas com prioridade e DLQ. ### 4. Semântico (Vector DB) Conhecimento, RAG, memória de longo prazo — busca por similaridade. **Regra de ouro:** cada banco faz o que sabe fazer. Misturar = desastre em produção. --- **3. As 4 camadas** ``

┌──────────┐ | AGENTE | (LangGraph) ┌──────────┐ |
└──────────┘ └──────────┘
┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐
└──────────┘ └──────────┘ └──────────┘ └──────────┘
Transacional Redis BullMQ Pinecone Postgres ``

`` **Por que separar:** performance, custo, manutenção, escala independente. --- **4. Case 1: afiliado que quebrou (Postgres pra tudo)** **Perfil:** Carlos, afiliado de finanças, 20k leads/mês **Stack inicial:** - 1 Postgres com tudo (usuários, sessões, mensagens, embeddings em JSONB) - 1 servidor (Hetzner, R\$ 100/mês) **Sintomas aos 3 meses:** - Latência: 1.5s (p95) — usuários reclamando - CPU: 95% — risco de queda - Storage: 80GB — JSONB gigante - Custos implícitos: R\$ 0 (mas cliente insatisfeito) **O que aconteceu:** 1. Sessões de conversa viraram **tabela** com 5M de rows 2. Embeddings em JSONB: queries de "similaridade" rodavam em Python no servidor 3. Tudo em uma query = contenção, lock, lentidão **Solução (3 semanas de trabalho):** 1. **Separar Postgres:** um para transacional, outro (lixo) para legados 2. **Migrar sessões para Redis:** TTL de 1h, latency caiu 80% 3. **Vector DB (Qdrant):** embeddings migrados, busca de 2s para 50ms 4. **Fila (BullMQ):** emails, webhooks, jobs de ML em background

****Resultado após refactor:**** - Latência p95: 1.5s → 200ms (7.5x mais rápido)
 - CPU: 95% → 25% - Custo: R\$ 100 → R\$ 350/mês (mas atende 5x mais usuários) - Receita recuperada: cliente não cancelou ****Lição:**** refactor de data stack é mais barato que perder clientes. --- ****5. Case 2: SaaS que escalou (data stack desde dia 1)**** ****Perfil:**** Ferramenta de marketing multi-tenant, lançada em 2024 ****Stack inicial (projetado desde o dia 1):**** - ****Postgres**** (Neon, serverless) — multi-tenant via `tenant\_id` + RLS - ****Redis**** (Upstash) — sessão, cache, rate limit - ****BullMQ**** (self-host) — jobs async, webhooks - ****Qdrant**** (Cloud free) — embeddings para RAG - ****Pino + Loki + Grafana**** — observabilidade ****Custo inicial:**** R\$ 0 (todos free tiers) ****Aos 6 meses (500 clientes):**** - 10k usuários ativos - 50k requests/dia - R\$ 200/mês total - Latência p95: 180ms - Uptime: 99.97% ****Aos 12 meses (2000 clientes):**** - 50k usuários ativos - 500k requests/dia - R\$ 800/mês total - Latência p95: 220ms (pico) - Uptime: 99.95% - Receita: R\$ 80k/mês (100x o custo de infra) ****Decisões-chave:**** 1. ****Multi-tenant com `tenant\_id`**** desde dia 1 (não schema por tenant) 2. ****RLS no Postgres**** desde dia 1 (não "vou adicionar depois") 3. ****Cache agressivo**** (TTL 5min em 80% das queries) 4. ****Embeddings gerados async**** (BullMQ job, não bloqueia request) 5. ****Logs estruturados**** (Pino) desde dia 1 (não printf) ****Lição:**** projetar data stack desde dia 1 evita refactor caro. --- ****6. Postgres para multi-tenant (RLS, schemas, row-level)**** ****3 estratégias:**** **###** 1. Tenant ID em cada linha (recomendado pra SaaS) `sql CREATE TABLE orders ( id UUID PRIMARY KEY, tenant_id UUID NOT NULL, ... ); ALTER TABLE orders ENABLE ROW LEVEL SECURITY; CREATE POLICY tenant_isolation ON orders USING (tenant_id = current_setting('app.current_tenant')::UUID);` **###** 2. Schema por tenant (pra high-value customers) Isolamento total, fácil backup por tenant. Migração é mais complexa. **###** 3. Database por tenant (pra compliance extremo) Isolamento máximo, LGPD-friendly, operacionalmente complexo. ****Recomendação:**** comece com (1), migre pra (2) só se cliente enterprise exigir. --- ****7. Redis: cache, sessão, rate limit**** ****Estrutura de chaves:**** `sess:user:{userId} → dados da sessão (TTL 1h) sess:conv:{conversationId} → histórico da conversa (TTL 24h) rate:user:{userId}:{action} → contador de ações (TTL 1h) lock:job:{jobId} → lock distribuído (TTL 5min) cache:api:{endpoint} → resposta cacheada de API (TTL 5min)` ****Quando usar Redis:**** - Estado temporário (< 24h) - Cache de leitura (read-heavy) - Rate limit - Dados que precisam de ACID (use Postgres) - Volume gigante sem TTL (custa caro) ****Free tier do Upstash:**** 10k comandos/dia. Suficiente pra começar. --- ****8. BullMQ: filas assíncronas, retry, DLQ**** `typescript import { Queue, Worker } from 'bullmq'; const emailQueue = new Queue('emails', { connection: { host: 'redis.example.com' } }, defaultJobOptions: { attempts: 3, backoff: { type: 'exponential', delay: 1000 }, removeOnComplete: 100, } }); await emailQueue.add('welcome', { userId: 'abc-123' }); const worker = new Worker('emails', async (job) => { const { userId } = job.data; await sendWelcomeEmail(userId); });` **\*\*Boas práticas:\*\*** - Sempre defina `attempts` (3 é um bom default) - Use `removeOnComplete` (não encher Redis) - Monitore com Bull Board - DLQ (Dead Letter Queue) pra jobs que falharam N vezes - Backoff exponencial (não martelar serviço externo) --- ****9. Vector DB: Pinecone vs Qdrant vs Weaviate**** | Feature | Pinecone | Qdrant | Weaviate |
 |-----|-----|-----|-----| | Tipo | Managed | Self/Cloud | Self/Cloud | |
 Custo (entry) | \$70/mês | Grátis | Grátis | | Latência p95 | ~50ms | ~20ms |

~30ms | | Multi-tenant | Namespaces | Collections | Classes | | Híbrido | Não
| Sim | Sim (nativo) | ****Recomendação:**** - ****POC**** (< 100k vetores): Qdrant
self-host - ****Produção pequena****: Qdrant Cloud (\$25/mês) - ****Produção
grande****: Pinecone Standard - ****Compliance****: Qdrant on-premise
****Embedding model recomendado:**** OpenAI `text-embedding-3-small`
(custo-benefício). --- ****10. Q&A com CTO + convidado Qdrant**** Sir. Nexus
Alencar + convidado da Qdrant responderão: - Como escolher entre Qdrant,
Pinecone e Weaviate em produção - Quando migrar de self-hosted para
managed - Como debugar latência no data stack - Quando Postgres +
tsvector substitui Vector DB - Custos reais em produção (R\$/mês por 1000
usuários) - RLS vs Schema vs Database para multi-tenant --- ***WB-2026-15 ·
Data Stack para Agentes IA · Janeiro 2027* *Por MMN AI-to-AI · 2026 ·
Licença: CC BY-SA 4.0* *"Agente sem data stack é arte. Agente com data
stack é produto."***